

Council Tax Explorer

CS551P Advanced Programming – Assessment 3 Design & Development Report

Student Name: [Your Name] | Render URL: <https://council-tax-explorer.onrender.com>

Application Overview

Council Tax Explorer is a Flask-based web application that enables users to browse, filter, search, and compare council tax data for all **296 billing authorities** in England across the 2025-26 and 2026-27 financial years. The data is open-government data published by MHCLG under the Open Government Licence v3.0, sourced from the annual CTR1 returns submitted by local authorities.

Data Source & Loading

Two ODS workbooks (Table 10, 2025-26 and 2026-27) each contain a *Data_Billing* sheet with 101 columns per authority. The **seed_db.py** script uses pandas (odf engine) to parse both files, cleans numeric fields, computes year-on-year percentage change, and loads the results into SQLite via Python's stdlib sqlite3. The process yields **592 council tax records** across 296 authorities. The database is regenerated on each deploy, ensuring fresh data.

Database Design

SQLite is used with **four tables**: *region* (9 rows), *authority_class* (5 rows), *authority* (296 rows), and *council_tax_record* (592 rows). *authority* references both *region* and *authority_class* by foreign key; *council_tax_record* references *authority*. Indexes are applied on *authority_id*, *year*, *region_code*, and *class_code* for query efficiency.

Pages & Templates

8 pages are implemented using Jinja2 template inheritance from a shared *base.html*: Home (national stats, league tables), Authorities (paginated/filtered list), Authority detail (per-LA KPIs, context comparisons, similar authorities), Regions overview, Region detail, Compare (up to 3 authorities side-by-side), Search, and 3 error pages (400/404/500). Composite templates are used throughout with a shared navigation bar and footer.

Error Handling

Flask `@app.errorhandler` decorators handle 400 (bad request – e.g. invalid compare IDs), 404 (missing authority or region), and 500 (server errors). The *authority_list* view validates sort keys against an allowlist before interpolating into SQL. SQL parameters are always passed as bound parameters to prevent injection. The seed script uses `try/except` on float conversion to safely skip non-numeric cells (marked [z] or [x] in the source data).

Testing

34 unit tests across 8 test classes cover every route (200/302/404), filter and search behaviour, the compare page, error handlers, and database integrity. Tests use Python's stdlib *unittest* with Flask's test client and a temporary SQLite database seeded with a single test authority. Tests are run with: `python3 -m unittest tests.test_app -v`.

Component	Implementation	Mark Criteria
Flask app + linked tables	4-table SQLite schema; authority FK→region, class; ctr FK→authority	10 pts
Open data display	Band D, area Band D, % change, taxbase, CT requirement per authority and year	10 pts
Data loading (592 records)	seed_db.py; pandas ODS→sqlite3; 296 authorities × 2 years	10 pts
Templates	8 pages + 3 error pages; base.html inheritance; composite layouts	10 pts
Error handling	400/404/500 handlers; allowlist SQL sort; parameterised queries	10 pts
Code structure	app.py, seed_db.py, templates/, static/, tests/ – clear separation	10 pts
Git	Initialised with feature branches (compare, regions, tests) and merges	10 pts
Tests	34 unittest tests; all routes, filters, DB integrity; no external deps	10 pts
Render deploy	gunicorn start command; seed in build step; requirements.txt	10 pts
Documentation	This report + README.md with install, test, maintenance, Render URL	10 pts